Environment Variable and Set-UID Lab

Shannon Tuohey

Lab Tasks

Task 1: Manipulating Environment Variables

- Use printenv or env command to print out the environment variables. Bash keeps a table on environment variables, and using these commands read the table and print it out, as shown.

```
stuohey@Ubuntu:~$ printenv PWD
/home/stuohey
stuohey@Ubuntu:~$ printenv HOME
/home/stuohey
stuohey@Ubuntu:~$ printenv USER
stuohey
stuohey@Ubuntu:~$ printenv PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin
:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
stuohey@Ubuntu:~$
```

- Export and unset are built in bash commands. I used export to include the new variable I created with the value “shannon” into the environment, and then unset to delete the variable from the environment.

```
stuohey@Ubuntu:~$ MY_VAR="shannon"
stuohey@Ubuntu:~$ echo $MY_VAR
shannon
stuohey@Ubuntu:~$ export MY_VAR
stuohey@Ubuntu:~$ printenv MY_VAR
shannon
stuohey@Ubuntu:~$ unset MY_VAR
stuohey@Ubuntu:~$ printenv MY_VAR
stuohey@Ubuntu:~$
```

Task 2: Passing Environment Variables from Parent Process to Child Process

- I compiled the following program, myprintenv.c, listed below. When compiled, it created a binary called a.out. I ran a.out and saved it to file. When that is run, it prints all environment variables available to the process. It has the same function as running printenv.

```
  GNU nano 8.4          myprintenv.c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

extern char **environ;

void printenv()
{
  int i = 0;
  while (environ[i] != NULL) {
     printf("%s\n", environ[i]);
     i++;
  }
}

void main()
{
  pid_t childPid;
  switch(childPid = fork()) {
    case 0:  /* child process */
      printenv();
      exit(0);
    default:  /* parent process */
      // printenv();
      exit(0);
  }
}
```

```
stuohey@Ubuntu:~/Downloads/Labsetup$ ./a.out > file
stuohey@Ubuntu:~/Downloads/Labsetup$ more file
SHELL=/bin/bash
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
QT_IM_MODULES=wayland;ibus
PTYXIS_PROFILE=3275d643bed634538b5ac394698247aa
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/gcr/ssh
MEMORY_PRESSURE_WRITE=c29tZSAyMDAwMDAgMjAwMDAwMAA=
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
GTK_MODULES=gail:atk-bridge
DBUS_STARTER_BUS_TYPE=session
PWD=/home/stuohey/Downloads/Labsetup
XDG_SESSION_DESKTOP=ubuntu
LOGNAME=stuohey
XDG_SESSION_TYPE=wayland
GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1
SYSTEMD_EXEC_PID=10897
XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.YVA3M3
HOME=/home/stuohey
USERNAME=stuohey
IM_CONFIG_PHASE=1
LANG=en_US.UTF-8
```

- I commented out the printenv() statement in the child process, and uncommented the printenv() statement in the parent process.

```
void main()
{
  pid_t childPid;
  switch(childPid = fork()) {
    case 0:  /* child process */
//      printenv();
      exit(0);
    default:  /* parent process */
     printenv();
      exit(0);
  }
```

- I compiled the program again and ran it, saving it into another file named file2. The output of this, file2, looks nearly identical to file. When I used the diff command, there were only a few minor differences. This is because when a process is created, it inherits the parent's environment variables. Therefore, both processes have access to the same environment. Minor differences appeared due to process specific variables.

```
stuohey@Ubuntu:~/Downloads/Labsetup$ diff file file2
21,22c21,22
< SYSTEMD_EXEC_PID=10897
< XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.YVA3M3
---
> SYSTEMD_EXEC_PID=2498
> XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.XJNZM3
32,33c32,33
< INVOCATION_ID=4655825b389347f49e7182d8624a6a60
< MANAGERPID=10875
---
> INVOCATION_ID=50daeea35249492a9aed5d5da9225a8d
> MANAGERPID=2454
43c43
< DBUS_STARTER_ADDRESS=unix:path=/run/user/1000/bus,gui
d=01175ce813e2a836944325df69c14b49
---
> DBUS_STARTER_ADDRESS=unix:path=/run/user/1000/bus,gui
```

Task 3: Environment Variables and execve()

- Execve() runs the new program in a calling process.
- Using the file myenv.c, as shown below, I compiled and ran it. When it ran, it printed nothing to the terminal. This is because the third argument in the execve() statement is NULL, passing no environment variables. However, when I changed the invocation of execve() to execve("/usr/bin/env", argv, environ), the environment variables printed. The environment must be explicitly passed to a new process.
- The new program gets the environment variables by being explicitly passed.

Listing 2: myenv.c

```c
#include <unistd.h>

extern char **environ;
int main()
{
  char *argv[2];

  argv[0] = "/usr/bin/env";
  argv[1] = NULL;
  execve("/usr/bin/env", argv, NULL);    ①

  return 0 ;
}
```

```
stuohey@Ubuntu:~/Downloads/Labsetup$ ./myenv
SHELL=/bin/bash
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
QT_IM_MODULES=wayland;ibus
PTYXIS_PROFILE=3275d643bed634538b5ac394698247aa
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/gcr/ssh
MEMORY_PRESSURE_WRITE=c29tZSAyMDAwMDAgMjAwMDAwMAA=
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
GTK_MODULES=gail:atk-bridge
DBUS_STARTER_BUS_TYPE=session
PWD=/home/stuohey/Downloads/Labsetup
XDG_SESSION_DESKTOP=ubuntu
```

Task 4: Environment Variables and system()

- Using the system() command differs from using execve() because it executes /bin/sh and asks the shell to execute the command. As shown by running this program, the environment variables of the calling process are passed to the new program /bin/sh.

```c
#include <stdio.h>
#include <stdlib.h>

int main(){
        system("/usr/bin/env");
        return 0;
}
```

```
stuohey@Ubuntu:~/Downloads/Labsetup$ ./system
LESSOPEN=| /usr/bin/lesspipe %s
USER=stuohey
XDG_SESSION_TYPE=wayland
PTYXIS_VERSION=49.1
SHLVL=1
HOME=/home/stuohey
OLDPWD=/home/stuohey/Downloads
DESKTOP_SESSION=ubuntu
GNOME_SHELL_SESSION_MODE=ubuntu
GTK_MODULES=gail:atk-bridge
MANAGERPID=2507
DBUS_STARTER_BUS_TYPE=session
SYSTEMD_EXEC_PID=2529
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus,g
uid=15d2a420a1442201623cd7fa69c477cf
```

Task 5: Environment Variables and Set-UID programs

- Set-UID allows the user to do many interesting actions, but it is risky because it escalates the user's privilege. When anyone runs the program, the user gains the root user's privilege.
- I created and compiled this program, and then changed the owner to root. Afterwards, I made it a set-UID program by using sudo chmod 4755 setuid.

```c
#include <stdio.h>
#include <stdlib.h>

extern char **environ;
int main()
{
  int i = 0;
  while (environ[i] != NULL) {
    printf("%s\n", environ[i]);
    i++;
  }
}
```

```
stuohey@Ubuntu:~/Downloads/Labsetup$ nano setuid.c
stuohey@Ubuntu:~/Downloads/Labsetup$ gcc -g setuid.c -o setui
d
stuohey@Ubuntu:~/Downloads/Labsetup$ sudo chown root setuid
[sudo: authenticate] Password:
stuohey@Ubuntu:~/Downloads/Labsetup$ sudo chmod 4755 setuid
stuohey@Ubuntu:~/Downloads/Labsetup$
```

- I then set three environment variables with the export command, PATH, LD_LIBRARY_PATH, and MY_VAR.

```
stuohey@Ubuntu:~/Downloads/Labsetup$ export PATH=$PATH:#HOME/
bin
stuohey@Ubuntu:~/Downloads/Labsetup$ export LD_LIBRARY_PATH=/
usr/local/lib
stuohey@Ubuntu:~/Downloads/Labsetup$ export MY_VAR="hello"
```

- When I ran the program, MY_VAR and PATH were outputted. However, LD_LIBRARY_PATH was not outputted. This was surprising to me, as I expected all three variables to be outputted. This does make sense, as LD_LIBRARY_PATH can change how programs load libraries. The fact that this did not output shows the security features of linux since a user could inject malicious code to this.

```
XDG_DATA_DIRS=/usr/share/ubuntu:/usr/share/gnome:/usr/local/s
hare/:/usr/share/:/var/lib/snapd/desktop
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:
/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin:/usr/loc
al/bin:#HOME/bin
GDMSESSION=ubuntu
```

```
IM_CONFIG_PHASE=1
LANG=en_US.UTF-8
MY_VAR=hello
LS_COLORS=rs=0:di=01;34:
1;35:bd=40;33;01:cd=40;3
:43:ca=00:tw=30:42:ow=34
```

Task 6: The PATH Environment Variable and Set-UID Programs

- When I attempted to compile the program below, an error was printed – "implicit declaration of function 'system'". I believe this may be a new security measure from Linux to prevent this potential threat. By attempting to change the variables, malicious users can control the behavior of the Set-UID program. So, at compile time, Linux prints out the error to prevent malicious activity.

```c
int main()
{
  system("ls");
  return 0;
}
```

```
stuohey@Ubuntu:~/Downloads/Labsetup$ gcc -g pathsetuid.c -o p
athsetuid
pathsetuid.c: In function 'main':
pathsetuid.c:2:9: error: implicit declaration of function 'sy
stem' [-Wimplicit-function-declaration]
    2 |         system("ls");
      |         ^~~~~~
stuohey@Ubuntu:~/Downloads/Labsetup$
```

Task 7: The LD PRELOAD Environment Variable and Set-UID Programs

- In this task I built a dynamic library. I used this program to basically override the sleep function. I then compiled the program using the following commands.

```c
#include <stdio.h>
void sleep (int s)
{
  /* If this is invoked by a privileged program,
     you can do damages here!   */
  printf("I am not sleeping!\n");
}
```

```
stuohey@Ubuntu:~/Downloads/Labsetup$ gcc -fPIC -g -c my
lib.c
stuohey@Ubuntu:~/Downloads/Labsetup$ gcc -shared -o lib
mylib.so.1.0.1 mylib.o -lc
```

- I then set the LD_PRELOAD environment variable.

```
stuohey@Ubuntu:~/Downloads/Labsetup$ export LD_PRELOAD=
./libmylib.so.1.0.1
```

- I compiled a program named myprog, as shown below. When it ran, it printed out "I am not sleeping!"

```c
/* myprog.c */
#include <unistd.h>
int main()
{
  sleep(1);
  return 0;
}
```

```
stuohey@Ubuntu:~/Downloads/Labsetup$ ./myprog
I am not sleeping!!
stuohey@Ubuntu:~/Downloads/Labsetup$
```

- I made the program's owner root and made it Set-UID. When I tried to run it, it printed nothing.

```
stuohey@Ubuntu:~/Downloads/Labsetup$ sudo chown root my
prog
stuohey@Ubuntu:~/Downloads/Labsetup$ sudo chmod 4755 my
prog
stuohey@Ubuntu:~/Downloads/Labsetup$ ./myprog
stuohey@Ubuntu:~/Downloads/Labsetup$
```

- However, when I changed to the root user and set the LD_PRELOAD environment variable as root, it worked and printed out!

```
root@Ubuntu:~# export LD_PRELOAD=./libmylib.so.1.0.1
root@Ubuntu:~# ./myprog
-bash: ./myprog: No such file or directory
root@Ubuntu:~# cd Downloads/Labsetup
-bash: cd: Downloads/Labsetup: No such file or director
y
root@Ubuntu:~# cd Downloads
-bash: cd: Downloads: No such file or directory
root@Ubuntu:~# cd /home/stuohey/Downloads/Labsetup
root@Ubuntu:/home/stuohey/Downloads/Labsetup# ls
a.out       libmylib.so.1.0.1  myprintenv.c  setuid.c
cap_leak.c  myenv              myprog        system
catall.c    myenv.c            myprog.c      system.c
file        mylib.c            pathsetuid.c
file2       mylib.o            setuid
root@Ubuntu:/home/stuohey/Downloads/Labsetup# ./myprog
I am not sleeping!!
root@Ubuntu:/home/stuohey/Downloads/Labsetup#
```

- I changed the owner of myprog to a different user named sally, then made the program Set-UID and set the LD_PRELOAD enviornment variable. When I tried to run it, it did not print out anything. This is because we are using the privilege of sally with my account, which disables LD_PRELOAD.
- To perform a little experiment, I edited the myprog program to this:

```
int main(){
        printf("Read UID: %d\n", getuid());
        printf("Effective UID: %d\n", geteuid());

        char *preload = getenv("LD_PRELOAD");
        if(preload)
                printf("LD_PRELOAD = %s\n", preload);
        else
                printf("LD_PRELOAD is not set\n");

        return 0;
}
```

- When I ran it normally, we can set that RUID=EUID, and the LD_PRELOAD is visible.

```
stuohey@Ubuntu:~/Downloads/Labsetup$ ./myprog
Read UID: 1000
Effective UID: 1000
LD_PRELOAD = ./libmylib.so.1.0.1
```

- When I changed the owner to root and made it Set-UID, RUID is not equal to EUID, and LD_PRELOAD is not set.

```
stuohey@Ubuntu:~/Downloads/Labsetup$ ./myprog
Read UID: 1000
Effective UID: 0
LD_PRELOAD is not set
```

- When I changed the owner to sally and made it Set-UID, again RUID is not equal to EUID, and LD_PRELOAD is not set.

```
stuohey@Ubuntu:~/Downloads/Labsetup$ ./myprog
Read UID: 1000
Effective UID: 1002
LD_PRELOAD is not set
```

- From this experiment, I can conclude that ANY Set-UID program (where effective UID ≠ real UID) disables `LD_PRELOAD`.

Task 8: Invoking External Programs Using system() versus execve()

- This file was written to ensure the user can read, but not write, onto the file that they type.

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
  char *v[3];
  char *command;

  if(argc < 2) {
    printf("Please type a file name.\n");
    return 1;
  }

  v[0] = "/bin/cat"; v[1] = argv[1]; v[2] = NULL;

  command = malloc(strlen(v[0]) + strlen(v[1]) + 2);
  sprintf(command, "%s %s", v[0], v[1]);

  // Use only one of the followings.
  system(command);
  // execve(v[0], v, NULL);

  return 0 ;
}
```

- I compiled the above program and made it root owned and Set-UID. The integrity of the system can be compromised. The system() command uses environment variables, which means a user can trick the program into running something else. I tested it by using an injection and writing “cat; ls” where the files should go. It actually created a new file named cat! This is a really dangerous vulnerability because it can allow unauthorized people to edit, add, or delete files.

```
stuohey@Ubuntu:~/Downloads/Labsetup$ ./catall "cat; ls'
rm /etc/passwd
a.out       libmylib.so.1.0.1  myprog.c
cap_leak.c  myenv              pathsetuid.c
cat         myenv.c            setuid
catall      mylib.c            setuid.c
catall.c    mylib.o            system
file        myprintenv.c       system.c
file2       myprog
stuohey@Ubuntu:~/Downloads/Labsetup$ ls
a.out       libmylib.so.1.0.1  myprog.c
cap_leak.c  myenv              pathsetuid.c
cat         myenv.c            setuid
catall      mylib.c            setuid.c
catall.c    mylib.o            system
file        myprintenv.c       system.c
file2       myprog
```

- When I commented the system() command out and used execve() instead, all the previous attacks stopped working. This is because the execve() command does not use the shell so it is not vulnerable to the same injection attack. Execve() directly executes a program, and since there is no program named “hack; ls” nothing happens and it cannot find that file.

```
stuohey@Ubuntu:~/Downloads/Labsetup$ ./catall "hack; ls
"
/bin/cat: 'hack; ls': No such file or directory (os err
or 2)
stuohey@Ubuntu:~/Downloads/Labsetup$
```

Task 9: Capability Leaking

- The Set-UID program can revoke privilege, but while doing this capability leaking can happen. If the program doesn't clean up capabilities when the privilege changes, they may still be accessible by non-privileged processes.
- I compiled the hack_leak program below and set the owner to root and made it Set-UID.

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>

void main()
{
  int fd;
  char *v[2];

  /* Assume that /etc/zzz is an important system file,
   * and it is owned by root with permission 0644.
   * Before running this program, you should create
   * the file /etc/zzz first. */
  fd = open("/etc/zzz", O_RDWR | O_APPEND);
  if (fd == -1) {
     printf("Cannot open /etc/zzz\n");
     exit(0);
  }

  // Print out the file descriptor value
  printf("fd is %d\n", fd);

  // Permanently disable the privilege by making the
  // effective uid the same as the real uid
  setuid(getuid());

  // Execute /bin/sh
  v[0] = "/bin/sh"; v[1] = 0;
  execve(v[0], v, 0);
}
```

- I then tried to exploit this vulnerability. I created a directory /etc/zzz and ensured that the directory was empty. Then, I ran the cap_leak program and wrote to /etc/zzz by writing "echo "hacked!!!" >&3". When I read the contents of /etx/zzz, it was successfully exploited and "hacked!!!" was written.

```
stuohey@Ubuntu:~/Downloads/Labsetup$ ./cap_leak
fd is 3
$ echo "hacked!!!!" >&3
$ exit
stuohey@Ubuntu:~/Downloads/Labsetup$ cat /etc/zzz
hacked!!!!
stuohey@Ubuntu:~/Downloads/Labsetup$
```